



Proyecto - Entrega 05

Informe Testing Proyecto

Ingeniería de Software 1

Integrantes

Tomás Mozo González

Daniel Ferney Gomez Rojas

Julian David Nieto Rodriguez

Diego Fernando Tique Ramirez

Tabla

Funcionalidad	Pruebas a realizar
Publicación exitosa de tareas	Publicar una tarea de paseo de mascotas. Publicar una tarea de limpieza del hogar. Publicar una tarea con pago decimal. Publicar una tarea con un monto elevado.
Validación de datos obligatorios	Título vacío. Descripción vacía. Categoría vacía. Categoría con espacios. Ubicación vacía.
Validación de reglas de negocio	Pago igual a cero. Pago negativo. Pago NaN. Pago Infinity. Tipo de pago inválido. Usuario igual a cero. Usuario negativo.

Pruebas Unitarias por Funcionalidad

Se implementaron pruebas unitarias para las principales funcionalidades de la aplicación. Las pruebas fueron desarrolladas por medio de la herramienta **Jest**, qué es un framework de pruebas que viene incluido por defecto en nuestro proyecto desarrollado con **NestJS** y **TypeScript**.

1.1 Publicación de tareas

Esta funcionalidad permite a los usuarios publicar nuevas tareas dentro de la plataforma, especificando información como título, descripción, categoría, ubicación, tipo de remuneración, monto ofrecido y usuario responsable de la publicación.

Casos de prueba implementados

Se implementaron pruebas unitarias utilizando Jest para verificar el correcto funcionamiento de la funcionalidad de publicación de tareas. Las pruebas se agruparon en tres comportamientos principales de la funcionalidad:

Publicación exitosa de tareas

Se verifica que el sistema permita crear correctamente una tarea cuando toda la información suministrada es válida.

Los escenarios evaluados fueron:

- Publicación de una tarea de paseo de mascotas.
- Publicación de una tarea de limpieza del hogar.
- Publicación de una tarea con remuneración decimal.
- Publicación de una tarea con remuneración de valor elevado.

En todos los casos se verificó que:

- La tarea fuera creada correctamente.
- Los espacios adicionales en los campos de texto fueran eliminados antes del almacenamiento.
- El servicio de persistencia (**Prisma**) recibiera la información esperada.

Validación de campos obligatorios

Se verificó que el sistema rechazara la creación de tareas cuando alguno de los campos obligatorios no contenía información válida.

Los escenarios evaluados fueron:

- Título vacío.
- Descripción vacía.
- Categoría vacía.
- Categoría compuesta únicamente por espacios.
- Ubicación vacía.

En cada uno de estos casos se comprobó que el sistema lanzara una excepción (**BadRequestException**) y que la tarea no fuera almacenada en la base de datos.

Validación de reglas de negocio

Se verificó el cumplimiento de las restricciones definidas por la lógica del negocio para la publicación de tareas.

Los escenarios evaluados fueron:

- Remuneración igual a cero.
- Remuneración negativa.
- Valor de remuneración no numérico (**NaN**).
- Valor de remuneración infinito (**Infinity**).
- Tipo de remuneración inválido.

- Identificador de usuario igual a cero.
- Identificador de usuario negativo.

En todos estos escenarios se verificó que la operación fuera rechazada y que no se realizara ningún intento de almacenamiento en la base de datos.

Casos límite evaluados

Las siguientes condiciones límite fueron evaluadas mediante pruebas unitarias:

- Publicación con monto igual a cero.
- Publicación con monto negativo.
- Publicación con valores especiales (NaN e Infinity).
- Publicación con tipo de remuneración no permitido.
- Publicación con campos obligatorios vacíos.
- Publicación con campos que contienen únicamente espacios en blanco.
- Publicación con identificadores de usuario inválidos.
- Publicación con remuneraciones decimales.
- Publicación con remuneraciones de gran magnitud.

```
const cases = [
  {
    name: 'User publishes a pet walking task',
    dto: {
      title: ' Walk the dog ',
      description: ' One hour walk ',
      category: ' Pets ',
      location: ' Bogotá ',
      paymentType: ' HOUR ',
      amount: 20,
      userId: 1,
    },
    shouldThrow: false,
  },
  {
    name: 'User publishes a house cleaning task',
    dto: {
      title: ' Clean the house ',
      description: ' Deep cleaning ',
      category: ' Home ',
      location: ' Medellín ',
      paymentType: ' DAY ',
      amount: 150,
      userId: 2,
    },
    shouldThrow: false,
  },
  {
```

Ejecución de las pruebas

Las pruebas unitarias fueron ejecutadas mediante el siguiente comando:

```
npx jest src/tasks/tasks.service.spec.ts
```

El resultado obtenido fue satisfactorio, ejecutándose correctamente la totalidad de los escenarios de prueba implementados, sin presentarse fallos.

```
PASS src/users/users.controller.spec.ts
PASS src/tasks/tasks.controller.spec.ts

Test Suites: 5 passed, 5 total
Tests:       23 passed, 23 total
Snapshots:   0 total
Time:        6.684 s, estimated 13 s
Ran all test suites.

dtique@LAPTOP-5RV0S8FV MINGW64 ~/Desktop/Universidad/ingenieria sistemas/quinto
semestre/ingeniería de software 1/monasteryy/project-backend (feature/testing-ta
sk-publication)
```

Coverage

Para calcular la cobertura de las pruebas se utilizó el comando:

```
npx jest src/tasks/tasks.service.spec.ts --coverage
```

Los indicadores obtenidos permiten evaluar el porcentaje del código ejercitado durante la ejecución de las pruebas unitarias implementadas para la funcionalidad de publicación de tareas.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	57.14	65.9	29.03	55.55	
src	43.33	50	75	37.5	
...controller.ts	100	75	100	100	6
app.module.ts	0	100	100	0	1-13
app.service.ts	100	100	100	100	
main.ts	0	0	0	0	1-17
src/prisma	42.85	100	0	40	
prisma.module.ts	0	100	100	0	1-8
...ma.service.ts	66.66	100	0	57.14	11-19
src/tasks	61.9	63.63	30.76	60	
...controller.ts	72.22	64.28	16.66	68.75	12,21,30,35,40
tasks.module.ts	0	100	100	0	1-13
tasks.service.ts	76.47	62.5	42.85	71.42	70-97
src/tasks/dto	100	100	100	100	
...e-task.dto.ts	100	100	100	100	
...e-task.dto.ts	100	100	100	100	
...tasks/entities	0	100	100	0	
task.entity.ts	0	100	100	0	1
src/users	52.77	75	16.66	50	
...controller.ts	72.22	75	16.66	68.75	12,17,22,27,32
users.module.ts	0	100	100	0	1-11
users.service.ts	54.54	75	16.66	44.44	9-32
src/users/dto	100	100	100	100	
...e-user.dto.ts	100	100	100	100	
...e-user.dto.ts	100	100	100	100	
...users/entities	0	100	100	0	
user.entity.ts	0	100	100	0	1
tasks.service.ts	76.47	62.5	42.85	71.42	70-97
src/tasks/dto	100	100	100	100	

Los resultados obtenidos para el servicio evaluado fueron:

- Statements: 76.47%
- Branches: 62.50%
- Functions: 42.85%
- Lines: 71.42%

1.2 Búsqueda y filtrado de tareas

Esta funcionalidad le permite a los usuarios consultar tareas publicadas disponibles aplicando diferentes filtros, tales como categoría, ubicación y remuneración mínima.

Casos de prueba implementados

1.) Filtrados básicos (Categoría, ubicación y remuneración mínima)

Se verifica que el sistema construya correctamente cada uno (por separado) de los filtros, generando la consulta correspondiente, a partir del siguiente código:

```

it('should filter tasks by category', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll('Mascotas');

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: 'Mascotas',
      location: undefined,
      amount: undefined,
    },
  });
});

it('should filter tasks by location', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll(undefined, 'Bogotá');

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: undefined,
      location: 'Bogotá',
      amount: undefined,
    },
  });
});

it('should filter tasks by minimum amount', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll(undefined, undefined, 30);

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: undefined,
      location: undefined,
      amount: {
        gte: 30,
      },
    },
  });
});

```

2.) Casos Límites Evaluados

Se ejecutaron las siguientes pruebas unitarias para casos límite:

- Categoría inexistente: Retorna lista vacía .
- Ubicación inexistente Retorna lista vacía.
- Consulta sin filtros: Realiza consulta sin aplicar filtrado.
- Aplicación simultánea de múltiples filtros: Construye consulta con combinación de filtros.

```
5 describe('TasksService', () => {
79   it('should handle non-existing location filter', async () => {
80     jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);
81
82     const result = await service.findAll(undefined, 'Antartida');
83
84     expect(prismaService.task.findMany).toHaveBeenCalledWith({
85       where: {
86         category: undefined,
87         location: 'Antartida',
88         amount: undefined,
89       },
90     });
91
92     expect(result).toEqual([]);
93   });
94
95   it('should return empty result when category does not exist', async () => {
96     jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);
97
98     const result = await service.findAll('CategoriaInexistente');
99
100    expect(prismaService.task.findMany).toHaveBeenCalledWith({
101      where: {
102        category: 'CategoriaInexistente',
103        location: undefined,
104        amount: undefined,
105      },
106    });
107
108    expect(result).toEqual([]);
109  });
110
111
112
113  it('should return all tasks when no filters are provided', async () => {
114    jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);
115
116    await service.findAll();
117
118    expect(prismaService.task.findMany).toHaveBeenCalledWith({
119      where: {
120        category: undefined,
121        location: undefined,
122        amount: undefined,
123      },
124    });
125  });
126});
```

```

it('should apply multiple filters simultaneously', async () => {
  jest.spyOn(prismaService.task, 'findMany').mockResolvedValue([]);

  await service.findAll(
    'Mascotas',
    'Bogotá',
    20,
  );

  expect(prismaService.task.findMany).toHaveBeenCalledWith({
    where: {
      category: 'Mascotas',
      location: 'Bogotá',
      amount: {
        gte: 20,
      },
    },
  });
});

```

Las pruebas mediante **Jest** se ejecutaron utilizando el siguiente comando en la terminal:

npx jest src/tasks/tasks.service.spec.ts

El resultado de las pruebas es el siguiente:

```

C:\Users\tomas\OneDrive\Documentos\UNAL\project-backend>npx jest src/tasks/tasks.service.spec.ts
PASS src/tasks/tasks.service.spec.ts
  TasksService
    ✓ should be defined (9 ms)
    ✓ should filter tasks by category (3 ms)
    ✓ should filter tasks by location (1 ms)
    ✓ should filter tasks by minimum amount (1 ms)
    ✓ should handle non-existing location filter (2 ms)
    ✓ should return empty result when category does not exist (2 ms)
    ✓ should return all tasks when no filters are provided (1 ms)
    ✓ should apply multiple filters simultaneously (1 ms)

Test Suites: 1 passed, 1 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       1.03 s, estimated 2 s
Ran all test suites matching src/tasks/tasks.service.spec.ts.

```

Como se puede ver de las ocho pruebas unitarias realizadas, todas fueron aprobadas sin ninguna falla.

3.) Coverage

Se utilizó el comando:

```
npx jest src/tasks/tasks.service.spec.ts --coverage
```

Para la generación de los indicadores de cobertura para la funcionalidad evaluada a partir de las pruebas unitarias realizadas. Como se puede ver a continuación:

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	7.26	8.62	4.76	5.96	
src	0	0	0	0	
app.controller.ts	0	0	0	0	1-10
app.module.ts	0	100	100	0	1-14
app.service.ts	0	100	0	0	1-6
main.ts	0	0	0	0	1-17
src/applications	0	0	0	0	
applications.controller.ts	0	0	0	0	1-32
applications.module.ts	0	100	100	0	1-11
applications.service.ts	0	0	0	0	1-39
src/applications/dto	0	100	100	0	
create-application.dto.ts	0	100	100	0	1-10
update-application.dto.ts	0	100	100	0	1-4
src/applications/entities	0	100	100	0	
application.entity.ts	0	100	100	0	1
src/prisma	42.85	100	0	40	
prisma.module.ts	0	100	100	0	1-8
prisma.service.ts	66.66	100	0	57.14	11-19
src/tasks	19.44	25	16.66	16.66	
tasks.controller.ts	0	0	0	0	1-40
tasks.module.ts	0	100	100	0	1-13
tasks.service.ts	63.63	83.33	33.33	55.55	11,35-48
src/tasks/dto	0	100	100	0	



Los resultados obtenidos para el servicio evaluado fueron:

- Statements: 63.63%
- Branches: 83.33%
- Functions: 33.33%
- Lines: 55.55%

1.3 Gestión de Postulaciones

1.4 Gestión de estados e historial de tareas

1.5 Verificación y seguridad de usuarios

